XSS y el navegador

Diapositivas de apoyo: Cross-site scripting, sus tres caminos y Mostrar texto. Evitar ejecutarlo. Ensayar a 125-145 palabras por minuto. Explicar los términos antes de señalar el fragmento técnico.

Una página también ejecuta instrucciones

Ya vimos lo que ocurre cuando una base de datos confunde valores con instrucciones. Ahora pensemos en lo que sucede al otro extremo: el navegador. Una página web combina contenido y código. HTML organiza lo que vemos, los estilos le dan apariencia y JavaScript puede responder a nuestras acciones. Esta capacidad permite formularios dinámicos y aplicaciones completas. Pero también obliga a controlar qué contenido llega a lugares donde el navegador lo interpreta como algo ejecutable. No toda información recibida debe tener esa capacidad.

Cross-site scripting, o XSS, ocurre cuando contenido no confiable llega a uno de esos contextos y se ejecuta dentro del sitio. Para decirlo de forma más sencilla: una persona consigue que el navegador de otra trate su entrada como código de la página. El problema afecta la confianza que el visitante tiene en ese sitio. No estamos hablando necesariamente de instalar un programa en la computadora ni de modificar todos los archivos del servidor. El contexto principal de ejecución es la página que la víctima está visitando.

Imaginemos una sección de comentarios. Ana publica un mensaje y Bruno abre la página para leerlo. La aplicación debería mostrar el comentario como contenido. Si, en cambio, inserta partes de ese mensaje de manera que puedan ejecutarse, el navegador de Bruno puede obedecer instrucciones aportadas por Ana. La aplicación hizo de intermediaria y dio al contenido una confianza que no debía tener. Por eso una entrada puede afectar a alguien distinto de quien la escribió. Esta diferencia ayuda a entender por qué XSS es un problema para los usuarios.

Tres maneras de llegar al navegador

La primera variante es XSS reflejado. El contenido llega en una petición y la respuesta lo devuelve de forma insegura. Podemos pensar en una búsqueda cuya página dice usted buscó esto. Si el valor se coloca mal en la respuesta, puede terminar interpretándose como código. Una dirección preparada podría llevar a una persona a esa respuesta. No basta con que una dirección tenga símbolos extraños para afirmar que hay XSS; debe existir un recorrido concreto desde la entrada hasta un contexto que permita la ejecución.

La segunda variante es XSS almacenado. Aquí el contenido queda guardado, por ejemplo en un comentario o un perfil. Después, la aplicación lo muestra a otras personas sin la protección apropiada. Es parecido a dejar una instrucción peligrosa pegada en un tablón donde todos esperan encontrar mensajes normales. La permanencia puede ampliar el alcance porque varios visitantes consultan el mismo contenido. Guardarlo en una base de datos no lo vuelve confiable. Cuando se recupera para mostrarlo, sigue necesitando un tratamiento correcto.

La tercera variante se basa en el DOM. DOM es el modelo de la página con el que trabaja JavaScript en el navegador. Podemos imaginarlo como una estructura de elementos que el programa puede leer y modificar. Si un script toma información de una dirección u otra fuente y la coloca en un destino inseguro, puede crear el problema del lado del cliente. En este caso debemos revisar también el código que modifica la página, no solamente las plantillas generadas por el servidor.

Qué puede pasar

Una consecuencia es manipular la interfaz. El usuario cree que está viendo información legítima, pero parte de la página fue cambiada. También podrían capturarse datos que escribe o realizarse acciones utilizando su sesión, según lo que permita la aplicación. Evitemos resumir XSS como robar cookies, porque esa frase deja fuera muchos efectos posibles. Además, una cookie con HttpOnly no se puede leer mediante el acceso normal de JavaScript a las cookies, pero eso no impide que un script malicioso interactúe con la página como la víctima.

Tampoco debemos pensar que una alerta emergente representa todo el daño. En el laboratorio utilizaremos una alerta inocua para demostrar que hubo ejecución. Es una señal visible y fácil de comparar, no la finalidad de un ataque real. Si la alerta aparece en una versión vulnerable y el mismo contenido se muestra como texto en la corregida, veremos una diferencia clara. La explicación debe centrarse en qué interpretó el navegador, no en lo llamativo de la ventana. Ese enfoque ayuda a identificar la causa y la solución.

Mostrar el contenido de forma segura

Una defensa fundamental es codificar la salida según el contexto. Codificar aquí significa representar caracteres de manera que se entiendan como texto y no cambien la estructura de la página. Por ejemplo, un símbolo que inicia una etiqueta HTML debe mostrarse como símbolo si forma parte de un comentario. No es lo mismo poner un valor entre texto HTML, dentro de una dirección o dentro de JavaScript. Cada lugar tiene sus reglas. Por eso la recomendación no es aplicar cualquier función de escape en cualquier parte.

En el navegador, si solamente queremos cambiar un texto, una opción segura es textContent. Su nombre expresa lo que hace: coloca contenido textual. Usar una operación que interpreta HTML sin necesidad aumenta los riesgos. Si el producto realmente necesita permitir formato HTML, hace falta una biblioteca de sanitización mantenida y una política de etiquetas y atributos admitidos. Sanitizar significa quitar o transformar construcciones peligrosas conforme a reglas. No conviene intentar resolverlo con una lista improvisada de palabras prohibidas o sustituciones aisladas.

En Flask, las plantillas HTML de Jinja escapan los valores de forma predeterminada. Podemos pasar un saludo como variable a una plantilla fija y permitir que se muestre como texto. El detalle importante es no desactivar esa protección con safe para contenido no confiable. También debemos evitar construir el código de la plantilla a partir de la entrada. Una plantilla fija que recibe un valor no es lo mismo que una entrada convertida en plantilla. Mantener esa separación reduce confusiones entre problemas del navegador y problemas del servidor.

Capas complementarias

Podemos resumir XSS con una comparación: en SQLi se altera lo que interpreta la base de datos; en XSS se altera lo que interpreta el navegador. Las defensas no se intercambian. Parametrizar SQL no evita un comentario ejecutable y escapar HTML no corrige una consulta construida por concatenación. Ahora veremos otro problema relacionado con el navegador: enviar una acción con una sesión válida sin que la persona haya tenido esa intención. Mi compañero explicará CSRF y la diferencia entre reconocer una sesión y comprobar una acción.

Apoyo para comprender y ensayar

Pronunciar XSS como equis, ese, ese. DOM puede explicarse como el árbol de elementos de una página. Origen es la combinación de esquema, host y puerto que delimita una parte de la confianza del navegador. No es necesario leer esta última definición al público si complica la explicación. Reservar una pausa breve entre las tres variantes y pedir que imaginen un comentario frente a un enlace de búsqueda. El laboratorio lo realiza el último expositor.

Diccionario de términos

HTML: Lenguaje que estructura el contenido de una página.

JavaScript: Lenguaje que puede ejecutar comportamientos dentro del navegador.

XSS: Ejecución de contenido no confiable dentro del contexto de una página.

DOM: Representación de los elementos de la página que JavaScript puede leer o modificar.

Escape de salida: Representación de caracteres para que se muestren como datos en el contexto correcto.

Sanitización: Limpieza de HTML según reglas para admitir formato y excluir construcciones peligrosas.

CSP: Política que restringe recursos y ejecuciones en el navegador; complementa otras defensas.

textContent: Propiedad que coloca texto sin interpretarlo como HTML.

Jinja: Motor de plantillas que Flask utiliza; escapa variables en plantillas HTML de forma predeterminada.